

ما هي الاختبارات البرمجية Software Testing وأهميتها وأهم أنواعها؟

أصبحت البرمجيات موجودة بشكل كبير في كل شيء حولنا وكل الأشياء التي نستخدمها ونعتمد عليها بشكل أساسي في تسيير حياتنا وأداء المهام والأنشطة المختلفة بشكل عام.

ومع ظهور البرمجيات وتطورها الكبير يوماً بعد يوم، إلى جانب انتشار المُبرمجين من مستويات احترافية متفاوتة، كان لابد من التأكد من جودة وسلامة هذه البرمجيات قبل الاعتماد عليها واستخدامها، خاصة في أنشطة ومهام حساسة.

من هنا جاءت الحاجة إلى الاختبار وظهر مصطلح اختبار البرمجيات، وأوجد المختبرين وانتشروا في مقابل انتشار المُبرمجين حول العالم.

لكن، ما المقصود بمصطلح اختبار البرمجيات؟ وما هي أهمية هذه الاختبارات؟ وما هي أهم أنواع الاختبارات البرمجية المعروفة؟

ما هو اختبار البرمجيات (Software Testing) ؟

اختبار البرمجيات أو الاختبارات البرمجية (Software Testing) هو مُصطلح يُطلق على العمليات التي تستهدف تقييم أي تطبيق برمجي تم تطويره من حيث الأداء والوظائف التي يؤديها وصُمم من أجلها. والغرض هنا هو التأكد من ومعرفة ما إذا كانت هذه البرمجيات تفي بالأغراض والمتطلبات المستهدفة منها أم لا.

وخلال عملية الاختبار أو التقييم التي يتولى مسئوليتها مُختبرين متخصصين، يتم الوصول والإشارة إلى العيوب التي يتضمنها التطبيق توجيه توصيات بإصلاحها؛ لضمان تقديم منتج برمجي عالي الجودة وخالي من أي عيوب تقنية.

وقد يكون الهدف من اختبار البرمجيات ليس فقط تنبيه المُبرمجين إلى عيوب تطبيقاتهم وإمدادهم بتوصيات واقتراحات إصلاحها، بل أيضاً إعطاء المستخدمين صورة كاملة عن جودة البرمجيات التي يستخدمونها.

وبناء عليه يمكننا تقسيم الأهداف من إجراء الاختبارات البرمجية إلى جزأين رئيسيين وهما:

1. **Validation:** هل التطبيق الذي قمنا بتطويره هو المنتج المستهدف والصحيح؟
2. **Verification:** هل يؤدي العنصر x ما وُضع من أجله؟ وقد يكون x جزء من الـ code أو واحد من المتطلبات أو نموذج وهكذا.

هكذا يبدو مفهوم اختبار البرمجيات والأهداف الأساسية التي وجد من أجلها هذا الإجراء وكذلك المُختبرين. لكن، ماذا عن أهميته؟ هذا ما سوف نكتشفه في السطور التالية...

ما أهمية اختبار البرمجيات أو الاختبارات البرمجية؟

توفر الاختبارات البرمجية معلومات واضحة ودقيقة عن جودة وأداء التطبيقات البرمجية المختلفة، بما يمنحنا القدرة على الحكم على مستقبل هذه البرمجيات وخطر فشلها. لذا تحمل عملية اختبار البرمجيات أهمية كبيرة وتؤدي أدواراً هامة بالنسبة لشركات البرمجيات أو على مستوى المستخدمين.

وفيما يلي مجموعة من أبرز فوائد الاختبارات البرمجية:

1. تحقيق رضا العميل وزيادة ولائه

جميع الشركات والعلامات التجارية بلا استثناء تسعى إلى تحقيق رضا العملاء والمستهلكين لديها؛ لأنه أمر من شأنه رفع درجة ولاء العميل للعلامة التجارية وبالتالي زيادة حجم المبيعات ومن ثم نسب الأرباح بما يخدم خطط التطور والنمو الخاصة بالشركة أو العلامة التجارية.

هذا هو بالضبط ما يحققه اختبار البرمجيات، إذ يتأكد من جودة المنتج قبل طرحه للبيع وقبل وصوله ليد العميل المستهدف، وبالتالي يُفوّت عليه احتمالية استرداد أمواله.

2. خفض التكاليف والمصروفات

فحص المنتج جيداً قبل طرحه للمستهلكين في السوق من شأنه خفض وتقليل التكاليف التي قد تتحملها الشركة أو المؤسسة في الصرف على ميزانية تسويق وترويج منتج غير سليم أو لا يؤدي الوظيفة التي تم إنتاجه من أجلها.

لذا فقط مصاريف التسويق، بل أيضاً تكاليف العمليات والأنشطة الخاصة بتجميع المنتج من العملاء أو استبداله من قبلهم وعمليات إصلاحه من جديد بعد استخدامه وهكذا.

3. تحسين العائد على الاستثمار

فائدة أخرى تؤديها الاختبارات البرمجية وهي العمل على تحسين العائد على الاستثمار الخاص بالشركات ورفعته إلى المستويات الأعلى، ذلك من خلال ضمان تقديم منتج عالي الجودة يحقق رضا العميل وخالي تماماً من أي مشاكل أو عيوب قد تُقلل من قيمته.

كان هذا جزء بسيط من أهمية اختبار البرمجيات وبعض الفوائد التي تُحققها الاختبارات البرمجية للشركات والمؤسسات وكذلك المستخدمين.

الآن، ننتقل لمحطة أخرى ذات شأن في هذا الموضوع وهي أنواع الاختبارات البرمجية المعمول بها والمنتشرة في أوساط شركات البرمجية والمُبرمجين.

ما هي أهم أنواع الاختبارات البرمجية؟

تضم الاختبارات البرمجية العديد من الأنواع، لكل نوع منها وظيفة معينة وهدف رئيسي نسعى للوصول إليه. ويمكن حصر أنواع اختبار البرمجيات في ما يلي:

1. اختبار الوحدة Unit Testing
2. اختبار التكامل Integration Testing
3. اختبار النظام System Testing
4. اختبار القبول Acceptance Testing
5. اختبار الرماز
6. اختبار التراجع Regression Testing

هذه هي المجموعة الأشهر لأنواع الاختبارات البرمجية، وفيما يلي شرح تفصيلي لمفهوم كل منها والوظيفة التي يؤديها كل نوع...

1. اختبار الوحدة Unit Testing

هذا النوع من الاختبارات يهتم فقط بالمفاهيم منخفضة المستوى والأجزاء البسيطة من النظام البرمجي. وفي هذا النوع يُجرى الاختبار ضمن الرماز البرمجي.

2. اختبار التكامل Integration Testing

اختبار التكامل تقوم وظيفته الأساسية على التأكد مما إذا كانت جميع أجزاء النظام تعمل بشكل سليم بعد دمجها معًا أم لا؟ وهو اختبار مهم للغاية؛ نظرًا لأن في الغالب يعمل أكثر من مُبرمج على كتابة رمز النظام.

3. اختبار النظام System Testing

أما هذا النوع من الاختبارات فإنه يهتم بالتأكد من أن جميع وظائف النظام الكبيرة تعمل جيدًا وتلبي جميع احتياجات المستخدم أو المستهلك.

4. اختبار القبول Acceptance Testing

هذا الاختبار عادة ما يقوم به المستخدم نفسه من أجل التأكد من أن التطبيق البرمجي أو النظام يلبي جميع احتياجاته وتطلعاته وهكذا.

يستخدم معظم منشئي منتجات البرمجيات عملية معالجة تدعى اختبار ألفا و اختبار بيتا للكشف عن الأخطاء حيث أن المستخدم النهائي فقط هو القادر على العثور على تلك الأخطاء.

The alpha test

يتم إجراء اختبار ألفا في موقع مطور البرمجيات بواسطة أحد العملاء أو الزبائن . يتم إجراء اختبارات ألفا في بيئة محكمة.

The beta test

اختبار beta على عكس اختبار alpha، مطور برمجيات غير موجود بشكل عام. ولذلك ، فإن اختبار beta هو تطبيق "مباشر" للبرنامج في بيئة لا يمكن لمطور البرمجيات التحكم فيها. وكنتيجة للمشكلات التي تم الإبلاغ عنها أثناء اختبارات beta، يقوم مهندسو البرمجيات بإجراء تعديلات ثم التحضير لإصدار منتج البرنامج للعملاء أو الزبائن.

5. اختبار الرماز

اختبار الرماز ينقسم إلى ثلاثة أنواع رئيسية من الاختبارات البرمجية، وهي كالتالي:

● اختبار الصندوق الأسود Black Box Testing

يتأكد من أن سلوك النظام مطابق تمامًا للمنصوص عليه في المتطلبات.

اختبار الصندوق الأسود ، الذي يسمى أيضًا الاختبار السلوكي يركز على المتطلبات الوظيفية للبرنامج.

إجراء الاختبارات باستخدام الصندوق الأسود ليس بديلاً عن الصندوق الأبيض. على العكس من ذلك ، إنه نهج تكميلي و من المرجح أيضا أن يكتشف فئة مختلفة من الأخطاء عن الطرق المستخدمة مع الصندوق الأبيض

تُستخدم اختبارات الصندوق الأسود لإثبات أن فئة البرمجيات تعمل بشكل صحيح ، وأن ادخال البيانات مقبول بشكل صحيح ويتم إنتاج المخرجات بشكل صحيح ، والحفاظ على سلامة المعلومات الخارجية (مثل قاعدة البيانات)

يحاول اختبار الصندوق الأسود العثور على أخطاء في الفئات التالية:

- (1) وظائف غير صحيحة أو مفقودة.
- (2) أخطاء في واجهة البرمجيات.
- (3) أخطاء في هياكل البيانات أو الوصول إلى قاعدة البيانات الخارجية.

(4) أخطاء سلوك أو أداء البرامجيات.

(5) أخطاء تهيئة الرامجيات و انهائها.

● اختبار الصندوق الرمادي Grey Box Testing

يتأكد كذلك من السلوك مطابق للمتطلبات مع وجود معرفة بسيطة ببنية النظام نفسه.

● اختبار الصندوق الأبيض White Box Testing

هذا النوع يقوم باختبار جميع أجزاء النظام ككل من تفرعات وعبارات وغيرها من أجزاء بنية النظام.

الصندوق الأبيض للبرمجيات يقوم على اساس الفحص الدقيق للتفاصيل الإجرائية. يتم اختبار المسارات المنطقية عبر البرنامج من خلال توفير حالات الاختبار التي تختبر مجموعات محددة من الشروط و / أو الحلقات. ويسمى أحياناً اختبار الصندوق الأبيض بالصندوق الزجاجي

يمكن لمهندس البرمجيات أن يستمد حالات الاختبار في الصندوق الأبيض كالآتي:-

(1) تضمن جميع المسارات المستقلة داخل الوحدة النمطية قد تم تمرينها مرة واحدة على الأقل.

(2) تمرين او تدريب جميع القرارات المنطقية على قراراتها بخصوص الصواب و الخطأ .

(3) تنفيذ جميع الحلقات .

(4) تمرين او تدريب هياكل البيانات الداخلية للتأكد من صحتها.

6. اختبار التراجع Regression Testing

اختبار التراجع يُقصد به إعادة اختبار النظام من جديد بعد إجراء أي تعديلات أو عمليات إصلاح للأخطاء التي تم العثور عليها في الاختبارات السابقة.

الخلاصة:

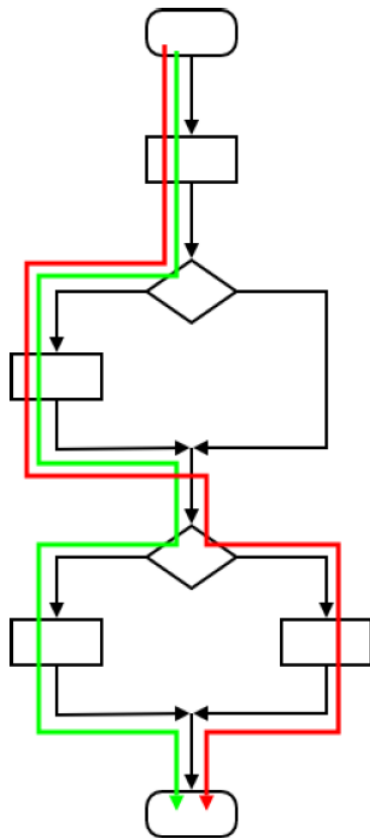
اختبار البرمجيات أصبح أمر ضروري لأي نظام برمجي أو تطبيق يتم تطويره، خاصة مع زيادة عدد البرمجيات التي يعتمد عليها المستخدمون في جميع مناحي حياتهم ويستخدمونها بشكل مستمر ويومي في معظم الأنشطة.

لذا ظهر مفهوم اختبار البرمجيات وانتشر المختبرين التقنيين حول العالم، لضمان سلامة أي من البرمجيات والتأكد من كونه يعمل بشكل صحيح ويحقق متطلبات المستخدمين وسلامتهم أيضاً.

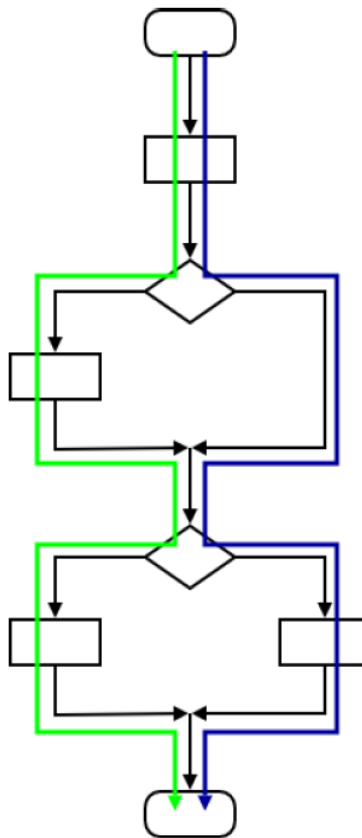
All Branches (Control Structure) Coverage-

All Paths Coverage-

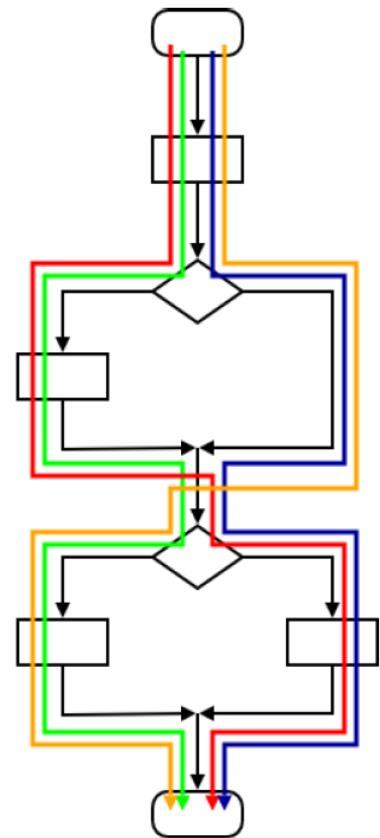
Statements Coverage



Statement Coverage
(aka line coverage)



Branch Coverage
(aka condition coverage)



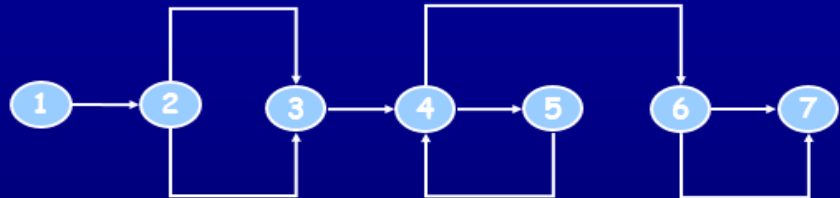
Path Coverage

Exponentiation Algorithm

```

1 scanf("%d %d",&x, &y);
2 if (y < 0)
    pow = -y;
    else
        pow = y;
3 z = 1.0;
4 while (pow != 0) {
    z = z * x;
    pow = pow - 1;
5 }
6 if (y < 0)
    z = 1.0 / z;
7 printf ("%f",z);

```

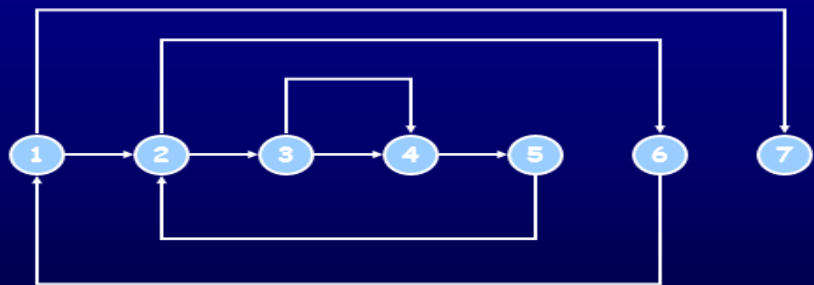


Bubble Sort Algorithm

```

1 for (j=1; j<N; j++) {
    last = N - j + 1;
2   for (k=1; k<last; k++) {
3     if (list[k] > list[k+1]) {
        temp = list[k];
        list[k] = list[k+1];
        list[k+1] = temp;
4     }
5   }
6 }
7 print("Done\n");

```



70

Cyclomatic Complexity

- Set of independent paths through the graph (basis set)
- $V(G) = E - N + 2$
 - E is the number of flow graph edges
 - N is the number of nodes

$V(G) = P + 1$, where P is the number of predicate nodes in the flow graph G —

A Flow Graph Example

```

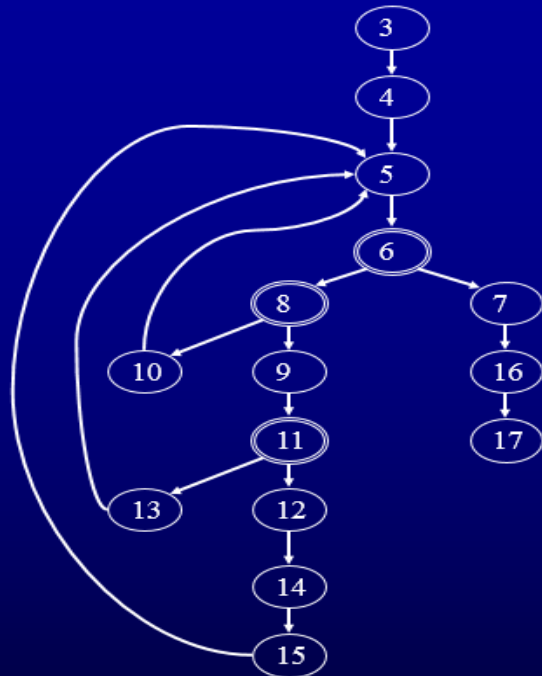
1  int functionY(void)
2  {
3      int x = 0;
4      int y = 19;

5  A: x++;
6      if (x > 999)
7          goto D;
8      if (x % 11 == 0)
9          goto B;
10     else goto A;

11  B: if (x % y == 0)
12     goto C;
13     else goto A;

14  C: printf("%d\n", x);
15     goto A;

16  D: printf("End of list\n");
17     return 0;
18  }
    
```



44

$$V(G)=17-15+2=4$$

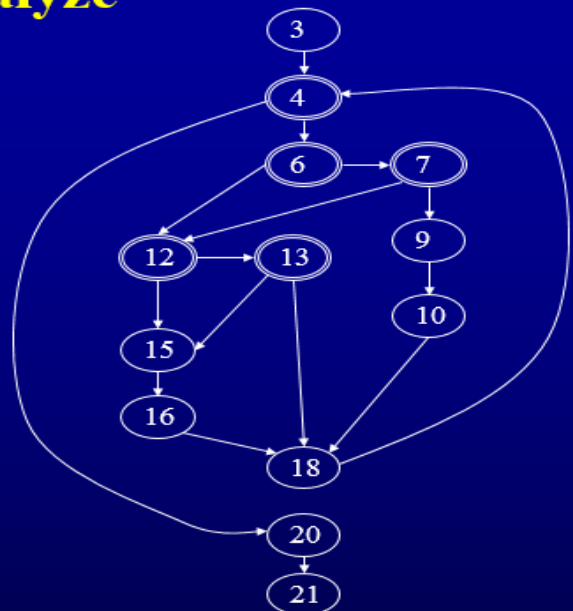
A Sample Function to Diagram and Analyze

```

1  int functionZ(int y)
2  {
3  int x = 0;

4  while (x <= (y * y))
5  {
6      if ((x % 11 == 0) &&
7          (x % y == 0))
8      {
9          printf("%d", x);
10         x++;
11     } // End if
12     else if ((x % 7 == 0) ||
13              (x % y == 1))
14     {
15         printf("%d", y);
16         x = x + 2;
17     } // End else
18     printf("\n");
19 } // End while

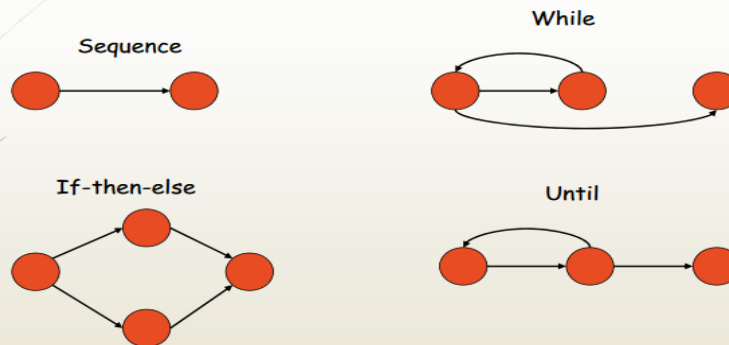
20 printf("End of list\n");
21 return 0;
22 } // End functionZ
    
```



46

$$V(G)=6$$

Flow Graph Notation



Cyclomatic Complexity & Testability

Complexity Number	Meaning
1-10	Structured and well written code Cost and Effort is less High Testability
10-20	Complex Code Cost and effort is Medium Medium Testability
20-40	Very complex Code Cost and Effort are high Low Testability
>40	Not at all testable Very high Cost and Effort

نموذج هولستد للقياس (١)

- يعتبر البرنامج سبيل لمجموعة من: العوامل و العمليات
- n_2, n_1 عدد العمليات و العوامل
 - N_2, N_1 التكرار الكلي للعمليات و العوامل
 - N اجمالي تكرار العوامل و العمليات
 - n_2^* عدد معالم المدخلات و المخرجات للخوارزمية

نموذج هولستد للقياس (٢)

استنادا لتلك البيانات اشتق هولستد مجموعة من المقاييس منها:

١. عدد المفردات اللغوية : $V = n1 + n2$
٢. طول البرنامج: $Len = n1 \log(n1) + n2 \log(n2)$
٣. حجم البرنامج: $Size = (\log(n1 + n2)) * N$
٤. الحجم الكامن للبرنامج: $V^* = (2 + n2^*) \log(2 + n2^*)$
٥. مستوى البرنامج: $L = V^* / V$
٦. صعوبة البرنامج: $D = V / V^*$

Ex. نموذج هولستد للقياس

العمليات	تكرار العمليات	العوامل	تكرار العوامل
#	1	<iostream.h>	1
include	1	Main	1
Void	1	X	3
()	1	5	1
{}	1		
Int	1		
;	3		
+	1		
>>	1		
<<	1		

بفرض الحاجة لتقويم البرنامج التالي المكتوب بلغة C++:

```
# include <iostream.h>
Void main () {
    int x ;
    cin >>x ;
    cout <<x+5 ;
}
```

$n1=10$ $Len=4\log 4+10\log 10$
 $n2=4$ $Size=18\log 14$
 $N1=12$ $n2^*=1$
 $N2=6$ $V^*=3\log 3$